

SEO PACKAGE

1. SEO Title

CHAR vs VARCHAR in SQL: How to Choose the Right Data Type Every Time (With CREATE TABLE Examples)

2. SEO Subtitle

A practical, beginner-friendly guide to SQL data types—so you design clean, efficient tables that quietly protect your data from bad values.

3. Featured Quote

"A data type isn't a formatting detail—it's a promise about what belongs in a column, and the database enforces that promise for free."

4. Meta Description

Learn CHAR vs VARCHAR in SQL, plus INT, DECIMAL, DATE, and BOOLEAN. Design clean, efficient tables with CREATE TABLE and pick the right data type every time.

(154 characters)

5. URL Slug

sql-char-vs-varchar-data-types-guide

6. Keywords

1. CHAR vs VARCHAR
2. SQL data types
3. CREATE TABLE SQL
4. VARCHAR vs CHAR performance

5. INT vs DECIMAL SQL
6. SQL schema design
7. SQL for beginners
8. database table design
9. DECIMAL data type SQL
10. BOOLEAN in SQL
11. DATE vs TIMESTAMP SQL
12. choosing SQL data types
13. SQL column types
14. relational database basics
15. SQL best practices
16. storing numbers as text
17. VARCHAR length limit
18. SQL tutorial
19. fixed vs variable length strings
20. coding interview SQL

7. Tags

SQL , Databases , CHAR vs VARCHAR , Data Types , CREATE TABLE , Schema Design , Backend Development , SQL Tutorial , Learn To Code , Coding Interview , Software Engineering , Relational Databases , Beginner Programming , Data Modeling

CHAR vs VARCHAR in SQL: Choosing the Right Data Type Every Time

8. Introduction

Every application you will ever build sits on top of a table. Before you can query, join, or index anything, someone has to *define* that table—and the single most consequential decision at that moment is choosing a **data type** for each column.

It sounds like a small, boring choice. It isn't. Pick the wrong type and you inherit a slow, buggy problem that only surfaces months later: numbers that sort in the wrong order, email addresses that get silently truncated, money that rounds incorrectly, storage that balloons for no reason. Pick the right

type and the database quietly does work *for* you—rejecting garbage input, sorting correctly, and keeping your schema lean.

This is also a classic **coding interview** topic. When an interviewer asks you to design a schema for users, orders, or students, they aren't testing whether you can type `CREATE TABLE`. They're testing whether you understand **why** `VARCHAR(255)` beats `CHAR(255)` for an email, why a phone number is text and not an integer, and why a price should never be a floating-point value. Getting these right signals that you think like an engineer who has been burned before—and learned from it.

By the end of this guide you'll be able to design clean, efficient schemas with confidence, and you'll understand the intuition behind the most common decision of all: **CHAR vs VARCHAR**.

9. Problem Overview

Let's state the real problem plainly, in our own words.

A **table** is the fundamental unit of storage in a relational database. Think of a paper form for enrolling new students. Each labeled blank line—*name*, *age*, *email*—is a **column**. Every student who fills out the form becomes a **row**. A table is just that form, turned into something a database can store, search, and reason about. Columns are the questions; rows are the answers.

You create a table with a single statement:

```
CREATE TABLE table_name (  
    column_name DATA_TYPE,  
    ...  
);
```

Every column is paired with a **data type**. That type is a constraint—a promise about what kind of value is allowed to live in that column. When you declare a column as an integer, the database will refuse to store the word `"hello"` there. The type is a gate: on every insert, the database asks *"Do you fit?"* If the value fits, it's stored; if not, it's turned away at the door.

Our task is to answer three questions confidently:

1. What are the common SQL data types, and what does each one mean?
2. **How do you decide** which type a column should be?
3. Specifically, when should you use **CHAR** versus **VARCHAR** for text?

10. Example

Here's a table for storing students, using the types we'll justify below:

```
CREATE TABLE students (  
  student_id INT,  
  full_name VARCHAR(100),  
  age INT,  
  email VARCHAR(255),  
  state_code CHAR(2),  
  balance_due DECIMAL(6, 2),  
  birth_date DATE,  
  created_at TIMESTAMP,  
  is_active BOOLEAN  
);
```

Let's explain **why** each choice is correct:

Column	Type	Why
student_id	INT	A whole-number identifier we count and order.
full_name	VARCHAR(100)	Names vary wildly in length—store only what's needed.
age	INT	We do math and sorting on it, so it must be a real number.
email	VARCHAR(255)	Emails vary and can be long; give them honest room.
state_code	CHAR(2)	Always exactly two letters—fixed length fits perfectly.
balance_due	DECIMAL(6,2)	Money needs exact fractions: 1999.99 , not 1999.9899 .
birth_date	DATE	We only care about the calendar day, not the time.
created_at	TIMESTAMP	We want the exact moment the row was created.
is_active	BOOLEAN	A simple on/off, true/false switch.

Notice `DECIMAL(6, 2)` : six total digits, two of them after the decimal point. That means the largest value is `9999.99` . So `1999.99` fits comfortably, but `12345.67` would not—read the precision carefully.

11. Intuition

Here's how an experienced engineer actually decides—and it has almost nothing to do with what a value *looks* like.

Name the real thing first, then let the type follow.

The trap is choosing a type based on appearance. A phone number *looks* numeric. So does a ZIP code. So does a product SKU. But ask the deeper question: *What will I actually do with this value?*

- You will never add two phone numbers together.
- Leading zeros matter in phone numbers and ZIP codes (`08540` is not `8540`).

- You won't compute the average of a SKU.

If you store a phone number as an integer, the leading zero vanishes and the value becomes meaningless. So a phone number is **text**, full stop—despite looking like a number.

Flip it around with age. Age is something you count and sort. If you lazily store ages as text ("9" , "90" , "100"), sorting them alphabetically puts "100" before "90" , because text sorts character by character— '1' comes before '9' . The moment you need real math or real ordering, text-numbers bite you.

So the guiding rule is behavioral, not visual:

If you'll count it, sum it, or sort it as a number—make it a number. Otherwise, it's probably text.

That single reframe—*ask what you'll do with it, not what it resembles*—resolves the vast majority of type decisions before CHAR vs VARCHAR even enters the picture.

12. Brute Force Solution

The "brute force" approach to schema design is: store everything as text (VARCHAR).

Beginners reach for this constantly. Every column becomes a string, because strings accept anything.

Algorithm 1. For every column, declare it VARCHAR(n) . 2. Store all values—numbers, dates, flags—as text. 3. Convert to the right type in application code whenever you need it.

Advantages - Dead simple: one type to remember. - Nothing ever gets rejected on insert—every value "fits."

Disadvantages - **Broken sorting:** numbers sort lexically ("100" before "90"). - **No math:** you can't SUM or AVG text without constant casting. - **No protection:** the database can no longer reject a value like "abc" in an age column—your gate is gone. - **Silent corruption:** dates and booleans stored as free text drift into inconsistent formats ("true" , "TRUE" , "1" , "yes"). - **Wasted space and slow queries:** no numeric or date indexing benefits.

Complexity - Design time: $O(1)$ (trivially fast to write). - Runtime cost: high and recurring—every query pays a casting and correctness tax forever.

This is the approach that *looks* easy today and costs you every day after.

13. Optimal Solution

The optimal approach is to **choose the smallest, most specific type that faithfully captures the meaning of the column**. Let's build the decision tree.

Step 1 — Numbers: INT vs DECIMAL

- Use **INT** for whole units you count: age, quantity, counts, IDs.
- Use **DECIMAL(p, s)** for exact fractions—especially **money**. **p** is total digits, **s** is digits after the decimal. **DECIMAL(6, 2)** stores up to **9999.99**.
- Avoid **FLOAT / REAL** for money—binary floating point can't represent **0.10** exactly, and cents go missing.

Step 2 — Text: CHAR vs VARCHAR

This is the headline decision.

	CHAR(n)	VARCHAR(n)
Storage	Fixed— pads every value to n	Variable—stores only what's there
Best for	Values that are <i>always</i> the same length	Values whose length <i>varies</i>
Space	Wastes space when values vary	Saves space; tiny per-value length overhead
Speed	Can be a hair faster for truly fixed data	Negligibly slower; wins nearly everywhere
Example	<code>state_code CHAR(2), country_code CHAR(2)</code>	<code>name, email, description</code>

The honest take: reach for VARCHAR about 90% of the time. **CHAR** earns its keep only for values that are genuinely a fixed length—two-letter state codes, ISO country codes, fixed-format currency codes. For general text, **CHAR** pads with wasted blank space and buys you almost nothing.

When in doubt, use VARCHAR.

Step 3 — Dates and Time: DATE vs TIMESTAMP

- **DATE** — just the calendar day. Perfect for a birthday.
- **TIMESTAMP** — date *plus* exact time. Perfect for "when was this row last touched?"

Step 4 — True/False: BOOLEAN

- **BOOLEAN** is a light switch: **TRUE / FALSE**, on/off. Don't model this as text **"yes"/"no"** or a stray **INT**.

Step 5 — Give text honest breathing room

A subtle but dangerous mistake: sizing `VARCHAR` too small. An `email VARCHAR(10)` looks fine until a real address—often 20–40+ characters—arrives and gets **truncated or rejected**. Nothing looks broken until someone's address is quietly mangled. A common safe default for emails is `VARCHAR(255)`.

The whole strategy, in one line: **pick the smallest type that captures what you actually mean—no smaller.**

14. Python Code

Data types are a database concept, but you'll define and enforce them from application code constantly. Here's a clean, production-quality **Python** example using the standard-library `sqlite3` module—no external dependencies—to create our schema, insert a valid row, and demonstrate how the wrong type causes real problems.

```
"""Demonstrate SQL data-type choices with Python's built-in sqlite3."""

import sqlite3

CREATE_STUDENTS = """
CREATE TABLE students (
    student_id    INTEGER PRIMARY KEY,
    full_name     VARCHAR(100) NOT NULL,
    age           INTEGER      NOT NULL,
    email         VARCHAR(255) NOT NULL,
    state_code    CHAR(2),
    balance_due   DECIMAL(6, 2) NOT NULL,
    is_active     BOOLEAN      NOT NULL
);
"""

def build_schema(conn: sqlite3.Connection) -> None:
    """Create the students table with well-chosen data types."""
    conn.execute(CREATE_STUDENTS)
    conn.commit()

def add_student(conn: sqlite3.Connection, name, age, email, state, balance, active):
    """Insert one student row. Types are enforced by column definitions."""
    conn.execute(
        """
        INSERT INTO students
            (full_name, age, email, state_code, balance_due, is_active)
        """
    )
```

```

VALUES (?, ?, ?, ?, ?, ?)
""",
    (name, age, email, state, balance, active),
)
conn.commit()

def top_balances(conn: sqlite3.Connection):
    """Return students ordered by balance – correct because age/balance are numeric."""
    return conn.execute(
        "SELECT full_name, balance_due FROM students ORDER BY balance_due DESC"
    ).fetchall()

def demo() -> None:
    conn = sqlite3.connect(":memory:")
    build_schema(conn)

    add_student(conn, "Asha Rao", 9, "asha.rao@example.com", "CA", 19.99, True)
    add_student(conn, "Ben Cohen", 90, "ben@example.com", "NY", 4.50, True)
    add_student(conn, "Cara Diaz", 100, "cara.diaz@example.com", "TX", 199.99, False)

    rows = top_balances(conn)
    assert rows[0][0] == "Cara Diaz", "highest balance should sort first"

    # Numeric ages sort as numbers: 9 < 90 < 100 (NOT '100' < '90' as text).
    ages = [r[0] for r in conn.execute(
        "SELECT age FROM students ORDER BY age ASC"
    ).fetchall()]
    assert ages == [9, 90, 100], f"numeric ordering broke: {ages}"

    print("All checks passed:", rows)
    conn.close()

if __name__ == "__main__":
    demo()

```

15. Code Walkthrough

- **CREATE_STUDENTS** — Our schema as a single SQL string. Each column pairs a name with a type: `age` is `INTEGER` (we'll sort and compare it), `email` is `VARCHAR(255)` (varies, can be long), `state_code` is `CHAR(2)` (always two letters), and `balance_due` is `DECIMAL(6, 2)` (exact money).
- **build_schema** — Executes the `CREATE TABLE` statement and commits. This is where the type "promises" are registered.

- **add_student** — Uses **parameterized queries** (`?` placeholders). This is not optional: it prevents SQL injection and lets the driver bind values with their proper types. Never build SQL by string concatenation.
- **top_balances** — Orders by `balance_due`. Because that column is numeric, `199.99` correctly ranks above `19.99`. If it were text, the ordering would be nonsense.
- **demo** — Inserts three students whose ages are `9`, `90`, and `100` —the exact values that expose the text-sorting bug. The `assert` confirms they sort as `[9, 90, 100]`, proving numeric typing did its job.

The `demo()` function is a runnable self-check: if the typing logic ever breaks, an `assert` fails loudly instead of silently returning wrong data.

16. Dry Run

Let's trace the ordering check step by step.

We insert three ages: `9`, `90`, `100`, stored in an `INTEGER` column.

Query: `SELECT age FROM students ORDER BY age ASC`

1. The database reads three integer values: `9`, `90`, `100`.
2. Because the column is numeric, it compares them **as numbers**: `9 < 90 < 100`.
3. Result: `[9, 90, 100]`. 

Now imagine the **brute-force** version, where age is `VARCHAR`:

1. Values stored as text: `"9"`, `"90"`, `"100"`.
2. Text comparison is character by character. Compare first characters: `'9'` vs `'9'` vs `'1'`.
3. `'1'` (ASCII 49) comes before `'9'` (ASCII 57), so `"100"` sorts first.
4. Result: `["100", "9", "90"]`.  Wrong.

Same data, different type, opposite outcome. The type is the correctness.

17. Complexity Analysis

Data-type selection isn't a runtime algorithm, but there are two real costs worth analyzing.

Time - Choosing a type is a **one-time design decision**—effectively $O(1)$ effort. - At runtime, correct types unlock **indexed** numeric and date comparisons (typically $O(\log n)$ lookups) instead of full scans

with per-row casting ($O(n)$ with a heavy constant). The wrong type quietly turns fast queries into slow ones.

Space - **CHAR(n)** always consumes space for n characters per row— $O(n)$ fixed, wasting $n - \text{length}$ bytes on every shorter value. - **VARCHAR(n)** consumes roughly the actual length plus a small length prefix— $O(\text{actual length})$. For variable data, this is strictly better.

Why these are correct: **CHAR** pads to full width by definition, so its cost is independent of the value; **VARCHAR** stores only real characters plus bookkeeping, so its cost tracks the data. That's exactly why **CHAR** wins only when every value truly is length n .

18. Alternative Solutions

- **TEXT / BLOB** — Some databases offer unbounded **TEXT** for large strings (articles, JSON). Prefer **VARCHAR(n)** when you know a sensible maximum, because the length limit is itself a data-quality guard. Reach for **TEXT** only when there is genuinely no reasonable bound.
- **ENUM (MySQL) / CHECK constraints** — For a fixed set of allowed strings (e.g., `status IN ('active', 'paused', 'closed')`), an **ENUM** or a **CHECK** constraint enforces the set at the database level—stronger than a bare **VARCHAR**.
- **NUMERIC vs DECIMAL** — In most databases these are synonyms; both give exact precision. Use either for money; the point is *exactness*, not the keyword.

Compared to plain **VARCHAR / INT**, these alternatives add stricter guarantees. Use them when the extra constraint reflects a real rule about your data.

19. Edge Cases

- **Empty input:** An empty string `''` is a *valid* **VARCHAR** value and is different from **NULL** (unknown/absent). Decide deliberately with **NOT NULL**.
- **Duplicates:** Types don't prevent duplicates—that's the job of **PRIMARY KEY** or **UNIQUE** (e.g., on `email`).
- **Minimum values:** `age = 0` is a valid **INT**; if negative ages are impossible, add a **CHECK** (`age >= 0`).
- **Maximum values:** A value larger than **DECIMAL(6, 2)**'s `9999.99`, or a string longer than **VARCHAR(n)**, is rejected or truncated. Size for the real-world maximum.
- **Leading zeros:** `08540` as an **INT** becomes `8540`. Zip codes and phone numbers must be text.

20. Common Mistakes

1. **Storing numbers as text.** Breaks sorting and math. If you'll count or order it numerically, make it a number.
2. **Sizing VARCHAR too small.** `email VARCHAR(10)` truncates real addresses silently. Give text honest room (e.g., `VARCHAR(255)` for emails).
3. **Using FLOAT for money.** Binary floating point loses cents. Use `DECIMAL(p, s)`.
4. **Using CHAR for variable-length text.** Pads with wasted blanks. Default to `VARCHAR` unless the length is truly fixed.
5. **Modeling booleans as free text.** `"yes"`, `"Y"`, `"1"`, `"true"` drift into chaos. Use `BOOLEAN`.
6. **Confusing DATE and TIMESTAMP.** A birthday doesn't need a time; a "last updated" moment does.

21. Interview Questions

- What's the difference between `CHAR` and `VARCHAR`, and when would you deliberately choose `CHAR`?
- Why should a phone number be stored as text rather than an integer?
- Why is `FLOAT` a poor choice for currency? What do you use instead?
- What does `DECIMAL(6, 2)` allow, and what's the largest value it can hold?
- How does column typing help sorting and indexing performance?
- What's the difference between an empty string `' '` and `NULL`?
- When would you pick `DATE` over `TIMESTAMP`?

22. Similar Problems

- **LeetCode 175 — Combine Two Tables:** Reinforces how well-typed columns join cleanly across tables.
- **LeetCode 595 — Big Countries:** Filtering on numeric columns (`area`, `population`)—only correct because those columns are numeric, not text.
- **LeetCode 627 — Swap Salary:** Uses a fixed-set text column (`sex`), a natural fit for `CHAR(1)` or an `ENUM`.
- **LeetCode 620 — Not Boring Movies:** Boolean-style filtering on odd IDs—mirrors why clean types make `WHERE` clauses trivial.

These are related because each depends on columns being modeled with the *right* type; the query is easy only because the schema was designed well.

23. Key Takeaways

- A table is a form: **columns are the questions, rows are the answers.**
- A **data type is a promise**—the database enforces it for free on every insert.
- **INT** to count, **DECIMAL** for money, **DATE/TIMESTAMP** for time, **BOOLEAN** for flags.
- For text: **default to VARCHAR** ; use **CHAR** only for genuinely fixed-length values like state codes.
- **Name the real thing first, then let the type follow.** Ask what you'll *do* with a value, not what it *looks* like.
- Get types right and everything downstream—sorting, math, storage, indexing—just works.

24. Watch the Video

Want to see these decisions made live, with the **CREATE TABLE** statement built column by column? Watch the full walkthrough here: **{LINK}**. Seeing the schema take shape—and watching a bad type get rejected at the door—makes the ideas stick far better than reading alone.

25. About the Series

This lesson is part of **Fun with Learning Technology**, a hands-on series that turns intimidating engineering topics into clear, intuition-first lessons. We move one concept at a time—no hand-waving, no jargon walls—building the mental models that let you reason about real systems. Whether you're preparing for a **coding interview**, leveling up your **software engineering** fundamentals, or just learning SQL properly for the first time, the series meets you where you are and takes you one solid step further each lesson.

26. Call To Action

If this made data types click for you:

- 👍 **Like** the video on YouTube—it genuinely helps the channel grow.
- 🔔 **Subscribe** so you never miss the next lesson in the series.
- 📄 **Subscribe to the Substack** for written deep-dives like this one.
- 💬 **Comment** with the trickiest type decision you've faced—phone numbers? money? dates?

-  **Share** this with a developer who's still storing numbers as text.

SOCIAL MEDIA

LinkedIn Post

Early in my career, I stored ages as text because I was lazy about data types. Then sorting put "100" before "90"—because text sorts character by character, not by value. That tiny shortcut became a real bug.

That's the thing about SQL data types: they look like a formatting detail, but they're actually a promise the database enforces for you. Mark a column as a number, and the database refuses to let a word slip in. You get correctness for free.

A few principles that have saved me repeatedly:

→ INT for things you count. DECIMAL for money (never FLOAT—it loses cents). → DATE for a birthday. TIMESTAMP for "when was this row last touched?" → BOOLEAN for true/false—not "yes"/"no" text that drifts into chaos.

And the big one—CHAR vs VARCHAR:

CHAR is a fixed-size box; it pads every value to full length. Great for a two-letter state code. VARCHAR is a stretchy box that stores only what it needs. For general text, VARCHAR wins about 90% of the time. When in doubt, use VARCHAR.

The deepest lesson isn't a syntax rule. It's a habit: name the real thing first, then let the type follow. A phone number looks numeric, but you never add two together and leading zeros matter—so it's text.

Don't ask what a value looks like. Ask what you'll actually do with it.

Get your types right, and everything downstream—sorting, math, storage, indexing—just works.

What's the trickiest type decision you've run into?

SQL #SoftwareEngineering #Databases
#LearnToCode #DataTypes

Twitter/X Thread

1/ Early on, I stored ages as text. Then sorting put "100" before "90". 🧑

The bug wasn't my logic. It was my data type. Here's how to pick the right SQL type every time 🧑

2/ First, the mental model. A table is just a paper form. The labeled blanks (name, age, email) are COLUMNS. Each person who fills it out is a ROW. Columns are the questions; rows are the answers.

3/ You build one with CREATE TABLE. Inside the parentheses, every column gets a data type sitting right next to it. That type is a promise about what's allowed to live there.

4/ Why it matters: a data type is like a shape sorter. The round hole only takes round pieces. Mark a column as a number and the database refuses to store a word. Free protection.

5/ Numbers: • INT → things you count (age, quantity) • DECIMAL → exact fractions like money
DECIMAL(6,2) = up to 6 digits, 2 after the dot. So 1999.99 fits perfectly. Never use FLOAT for money—it loses cents.

6/ The big one—CHAR vs VARCHAR: • CHAR = fixed-size box, pads to full length. Great for a 2-letter state code. • VARCHAR = stretchy box, stores only what it needs.

7/ Honest take: VARCHAR is what you'll reach for ~90% of the time. CHAR earns its keep only for truly fixed things (country codes). Unsure? Use VARCHAR.

8/ Dates & flags: • DATE → just the day (birthday) • TIMESTAMP → date + exact time (last updated) • BOOLEAN → a light switch, true/false Pick the smallest type that captures what you mean.

9/ The real skill: name the thing first, then let the type follow. A phone number LOOKS numeric—but you never add two together and leading zeros matter. So it's text. Ask what you'll DO with it, not what it looks like.

10/ Get your types right and everything downstream—sorting, math, storage—just works. Full walkthrough (Lesson 20) 🧑 Like + follow for the next one.

Facebook Post

Ever had numbers sort in a weird order—like "100" showing up before "90"? 🧑 There's a good chance the culprit is a data type.

In SQL Lesson 20, we build the very first thing every database needs: a table. And we tackle the decision that trips up almost every beginner—CHAR vs VARCHAR.

The short version:  INT for counting, DECIMAL for money  CHAR for fixed-length text (like a 2-letter state code)  VARCHAR for everything else (names, emails)  When in doubt? Use VARCHAR.

The golden rule: don't ask what a value *looks* like—ask what you'll actually *do* with it. (That's why a phone number is text, not a number!)

Come learn how to design clean, efficient tables with confidence. 

Reddit Summary

CHAR vs VARCHAR, and how to actually choose SQL data types

A quick writeup of the decision-making behind SQL data types, aimed at people who are past "hello world" but still unsure why their schema choices matter.

The core idea: a data type is a constraint the database enforces for you. Declare a column numeric, and it rejects non-numbers on insert—so you get validation for free.

Quick reference: - **INT** for whole units you count; **DECIMAL(p, s)** for money (avoid FLOAT—binary floating point drops cents). - **DATE** for a calendar day; **TIMESTAMP** when you need the exact time too. - **BOOLEAN** for true/false instead of ad-hoc "yes"/"1" text. - **CHAR(n)** pads every value to length n—only worth it for genuinely fixed-length data (state codes, ISO country codes).

VARCHAR(n) stores just what's there and is the right default for general text.

Two common bugs worth internalizing: 1. Storing numbers as text breaks ordering ("100" sorts before "90" lexically) and blocks real math. 2. Sizing VARCHAR too small (e.g. email VARCHAR(10)) silently truncates real values.

The heuristic that ties it together: **name the real thing first, then let the type follow**. A phone number looks numeric but you never sum two of them and leading zeros matter—so it's text. Ask what you'll do with a value, not what it resembles.

Happy to discuss where CHAR still genuinely beats VARCHAR in your experience.

GITHUB README

```
# SQL Data Types – CHAR vs VARCHAR (Lesson 20)
```

```
A beginner-friendly guide to designing clean, efficient SQL tables with the  
`CREATE TABLE` statement and the right data type for every column.
```

Problem

Every database starts with a table, and every column in that table needs a **data type**. The type is a promise about what kind of value is allowed to live in the column—and the database enforces it for you on every insert. Choosing well means correct sorting, exact math, and lean storage. Choosing poorly means silent truncation, broken ordering, and lost precision.

This lesson focuses on the decision beginners get wrong most often:
CHAR vs VARCHAR.

Intuition

Don't choose a type based on what a value *looks* like. Choose based on what you'll *do* with it.

- A phone number looks numeric, but you never add two together and leading zeros matter → it's **text**.
- Age is counted and sorted numerically → it's a **number**.

> Name the real thing first, then let the type follow.

Approach

Need	Type	Notes
Count whole units	<code>INT</code>	age, quantity, IDs
Exact fractions / money	<code>DECIMAL(p, s)</code>	<code>DECIMAL(6,2)</code> → up to <code>9999.99</code>
Fixed-length text	<code>CHAR(n)</code>	state codes, country codes
Variable-length text	<code>VARCHAR(n)</code>	names, emails – the ~90% default
Calendar day	<code>DATE</code>	birthdays
Day + exact time	<code>TIMESTAMP</code>	"last updated"
True / false	<code>BOOLEAN</code>	on/off flags

CHAR vs VARCHAR: `CHAR` pads every value to full length (fast, but wastes space when values vary). `VARCHAR` stores only what's there (tiny per-value overhead, wins nearly everywhere). When in doubt, use `VARCHAR`. Give text columns honest room—an `email VARCHAR(10)` will silently truncate real addresses.

Python Solution

Uses only the standard library (`sqlite3`).

```
\\python
import sqlite3

CREATE_STUDENTS = """
CREATE TABLE students (
    student_id    INTEGER PRIMARY KEY,
    full_name     VARCHAR(100) NOT NULL,
    age           INTEGER      NOT NULL,
```



```

    email        VARCHAR(255) NOT NULL,
    state_code    CHAR(2),
    balance_due   DECIMAL(6, 2) NOT NULL,
    is_active     BOOLEAN      NOT NULL
);
"""

def demo():
    conn = sqlite3.connect(":memory:")
    conn.execute(CREATE_STUDENTS)
    conn.executemany(
        "INSERT INTO students (full_name, age, email, state_code, balance_due, is_active)"
        " VALUES (?, ?, ?, ?, ?, ?)",
        [
            ("Asha Rao", 9, "asha@example.com", "CA", 19.99, True),
            ("Ben Cohen", 90, "ben@example.com", "NY", 4.50, True),
            ("Cara Diaz", 100, "cara@example.com", "TX", 199.99, False),
        ],
    )
    ages = [r[0] for r in conn.execute("SELECT age FROM students ORDER BY age").fetchall()]
    assert ages == [9, 90, 100], f"numeric ordering broke: {ages}"
    print("Ages sorted numerically:", ages)
    conn.close()

if __name__ == "__main__":
    demo()
\\`\\`\\`

Run it:

\\`\\`\\`bash
python students_demo.py
\\`\\`\\`

## Complexity

- Design cost: one-time,  $O(1)$ .
- Runtime: correct types enable indexed numeric/date lookups ( $\sim O(\log n)$ ) instead of full scans with per-row casting ( $O(n)$ ).
- Storage: CHAR(n) uses  $O(n)$  per row (padded); VARCHAR(n) uses  $O(\text{actual length})$  plus a small length prefix.

## Video

 Watch the full walkthrough: {LINK}

## Article

 Read the complete guide with dry runs, edge cases, and common mistakes: part of the Fun with Learning Technology series.
```

Skipped: nothing — this was a prose deliverable, not code, so the ponytail ladder doesn't apply. The one runnable Python self-check (`demo()` with `assert`) is included per spec.

Quick Review

When do you reach for CREATE TABLE and explicit data types?

When modeling a new domain entity or formalizing storage rules — CREATE TABLE defines the schema, columns, and type-based validation for a relational table.

How does column type choice affect performance/storage?

Right types shrink row size and speed queries; INT beats storing numbers as text, and fixed CHAR vs variable VARCHAR trades padding for flexibility.

Space: CHAR(n) vs VARCHAR(n) storage cost?

CHAR(n) always reserves n chars (padded), wasting space for short values. VARCHAR(n) stores only actual length plus a small length prefix.

Common bug: what breaks when numbers are stored as text?

Sorting and comparisons go lexicographic — '10' < '9', arithmetic fails, indexes bloat. Store numeric values in INT/DECIMAL instead.

INT vs DECIMAL — when to use each?

INT for whole counts/IDs. DECIMAL(p,s) for exact fractional values like money, avoiding FLOAT's rounding errors.

Interview: why can VARCHAR too small cause data loss?

Inserts exceeding the length get truncated or rejected. Size VARCHAR to the realistic max the domain needs, with headroom.

sql Lesson 20: CREATE TABLE and Data Types (The CREATE TABLE statement defines the schema and structural integrity of a relational database. Choosing appropriate data types ensures storage efficiency, query performance, and consistent validation of incoming information. You reach for these definitions whenever you need to model a new domain entity or formalize the storage requirements for a specific dataset.) — free Python cheat sheet